CHAPTER 24

Prompt Evaluation and Regression Suites

In this chapter

Introduction: Nineteen Ways to Be Wrong . . . 3
24.1 Success Criteria Before Anything Else . . . 4
24.2 An Evaluation Set Is Not a Regression Suite . . . 5
24.3 Designing the Set: the Three Documented Principles . . . 5
24.4 Grading: Three Methods, Ranked by the Documentation . . . 6
24.5 What the Six Graders Say . . . 7
24.6 How Big Must the Set Be? . . . 8
24.7 The Gate, and What It Actually Does . . . 9
24.8 The Winner's Curse . . . 9
24.9 What a Model-as-Judge Costs You . . . 10
24.10 Visual Analogies . . . 11
24.11 Architecture: The Suite as a Pinned Artefact . . . 12
24.12 Examples . . . 13
24.13 Code Walkthrough: A Gate That Knows Its Own Error Rates . . . 13
24.14 Mini Project: The Evaluation Harness and Regression Gate . . . 14
24.15 Panel A — Six Graders, One Set of Responses . . . 15
24.16 Panel B — How Big the Set Must Be . . . 15
24.17 Panel C — Three Gates, Four Sizes, Ten Thousand Replications Each . . . 16
24.18 Panel D — The Winner's Curse . . . 17
24.19 Panel E — What an Imperfect Judge Costs . . . 17
24.20 Panel F — What the Suite Costs, and What It Is Worth . . . 18
24.21 Panel G — The Suite Is an Artefact . . . 18
24.22 Hands-on Labs . . . 19
24.23 Lab Solutions . . . 19
24.24 Best Practices . . . 20
24.25 Common Mistakes . . . 22
24.26 Performance Considerations . . . 24
24.27 Security Considerations . . . 24
24.28 Real Production Examples . . . 25

24.29 Certification Notes 25
24.30 Twenty Practice Questions 26
24.31 Answers and Explanations 27
24.32 Ten Practical Exercises 28
24.33 Five Challenge Labs 28
24.34 Ten Interview Questions 28
24.35 Summary 29
24.36 Glossary 30
24.37 Further Reading 31

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Write success criteria that are specific, measurable, achievable and relevant, and say why "the model should classify sentiments well" is none of those.
2. Choose among the three documented grading methods — code-based, human, and model-based — and justify the choice by speed, reliability and scale rather than by taste.
3. Distinguish an **evaluation set**, which measures capability, from a **regression suite**, which pins behaviour, and design each differently.
4. Size an evaluation set for a stated difference using a power calculation, not a round number.
5. Use a **paired** design and know exactly how many examples it saves over two independent samples.
6. Recognise the **winner's curse** when you compare many prompt variants, and correct for it.
7. Quantify what an imperfect model-as-judge does to both the level and the *difference* you are trying to measure.
8. Build a regression gate whose false-alarm rate and detection rate are both known numbers rather than hopes.

NOTE

Prerequisites

Chapter 8 for the Wilson interval, for Cohen's kappa, and for why accuracy on a rare event is a bad safety metric. **Chapter 10** for the triage that decides whether a failure is a prompting problem at all. **Chapter 12** for the sample size a reported gap requires, and for the honesty of publishing a hypothesis that did not survive its own confidence intervals. **Chapter 15** for evaluation-set design as a purchasing decision, and for the measured **$23,179 a month** that a 2,000-example set was worth over a 50-example one. **Chapter 20** for the prompt as parts with a kind and a stability. **Chapter 23** for conformance and correctness as separate measurements.

Guardrails, validation and prompt debugging — what you do once the suite has gone red — are **Chapter 25**. Observability and tracing in production, where the eval set is replaced by live traffic, are **Chapter 57**.

Introduction: Nineteen Ways to Be Wrong

A team ran a prompt bake-off. Twenty variants of a triage prompt, each scored on the same 200-example set, best one wins. Variant 14 came out on top at 83.5 per cent against a 79.5 per cent incumbent — four points, a clear winner, shipped on the Thursday.

It did not hold. The next month's traffic looked like the incumbent's, and the eval set said 80.1 per cent when they re-ran it on fresh cases.

Section 24.20 reproduces the whole thing with 5,000 replications, and the result is not that variant 14 was unlucky. It is that **variant 14 could not have been anything else**. When twenty prompt variants all have *exactly the same* true accuracy of 80 per cent and each is scored on 200 examples, the best of them averages **83.82 per cent** — **3.82 points of pure selection noise**, with no prompt being better than any other. On a fresh set the same winner scores **79.96 per cent**: the entire margin evaporates. The winner is

the best again only **4.9 per cent** of the time, which is chance for twenty candidates. And Chapter 12's rule — do the Wilson intervals fail to overlap? — declares the winner genuinely better than variant 0 in **0.2 per cent** of replications. The statistics were right. Nobody ran them.

That is the first of the four failures this chapter is built around. The second is that the team's regression gate — 50 examples, "fail if accuracy drops" — would have caught a real five-point regression **16.6 per cent** of the time while firing spuriously **42.7 per cent** of the time when nothing had changed at all. The third is that switching from an exact-match grader to a model-as-judge with 92.6 per cent agreement and a Cohen's kappa of 0.759 does not merely add noise: it **shrinks a real five-point difference to 4.16 points**, by a factor you can calculate in advance, and costs you 45.2 per cent more examples to see what remains. And the fourth is the cheapest of all: on the same twenty-four responses, six defensible graders return **100.0, 75.0, 62.5, 62.5, 62.5 and 50.0 per cent**, and the difference between the highest and the lowest is a choice somebody made in an afternoon.

The documentation's framing is the right place to start, and it is disarmingly plain: *"Building a successful LLM-based application starts with clearly defining your success criteria and then designing evaluations to measure performance against them. This cycle is central to prompt engineering."* Chapter 20 added the corollary that matters most — **not every failing eval is a prompting problem**. This chapter adds the one that comes before it: **not every failing eval is a failure**, and not every passing one is a pass.

24.1 Success Criteria Before Anything Else

The documentation gives four properties of a good success criterion and a worked contrast for each. They are worth taking literally, because each rules out a specific bad habit.

Specific. *"Instead of 'good performance,' specify 'accurate sentiment classification.'"* The bad habit is a criterion so general that no result can contradict it.

Measurable. *"Use quantitative metrics or well-defined qualitative scales."* The documentation's example is the one to remember, because it takes the hardest possible case and quantifies it anyway. Bad: *"Safe outputs."* Good: *"Less than 0.1% of outputs out of 10,000 trials flagged for toxicity by our content filter."* Note what that criterion contains that the bad one does not: a rate, a denominator and a measurement instrument. Chapter 8's arithmetic then tells you that 0.1 per cent on 10,000 trials means roughly ten events, and that a Wilson interval on ten events out of 10,000 runs from about 0.05 to 0.18 per cent — so a target of 0.1 per cent is measurable at that sample size and a target of 0.01 per cent is not.

Achievable. *"Base your targets on industry benchmarks, prior experiments, AI research, or expert knowledge. Your success metrics should not be unrealistic to current frontier model capabilities."* Chapter 10's triage is the instrument here: a criterion that requires exact arithmetic on unbounded inputs is not a hard target, it is a structurally impossible one, and no amount of prompting will reach it.

Relevant. *"Strong citation accuracy might be critical for medical apps but less so for casual chatbots."*

The documentation's fully worked example is the best single paragraph in the source material, and it repays reading twice:

> *"Our sentiment analysis model should achieve an F1 score of at least 0.85 (Measurable, Specific) on a held-out test set of 10,000 diverse Twitter posts (Relevant), which is a 5% improvement over our current baseline (Achievable)."*

Eight documented dimensions are offered as a non-exhaustive list of what to write criteria about: **task fidelity, consistency, relevance and coherence, tone and style, privacy preservation, context utilisation, latency and price**. The last two are the ones engineers under-write. Chapter 11 gave you a latency model and Chapter 19 gave you a price; both belong in the success criteria alongside accuracy, because a prompt that is two points better and 40 per cent slower has not obviously won. The documentation states the general case directly: *"Most use cases need multidimensional evaluation along several success criteria."*

24.2 An Evaluation Set Is Not a Regression Suite

These two artefacts get the same name in most codebases and they answer different questions.

DEFINITION

Evaluation set

A sample from your real task distribution, used to **measure a capability** and to compare candidates. Its job is to produce a number with an interval on it. It is refreshed when the task distribution changes, and its cases may be replaced freely.

DEFINITION

Regression suite

A **pinned** set of cases with a pinned grader and a pinned decision rule, used to detect that behaviour which used to be right has stopped being right. Its job is to produce a **PASS or FAIL**, not a number. Its cases must not change without a version bump, because changing them changes the meaning of "pass".

	Evaluation set	Regression suite
Question	how good is it?	has it got worse?
Output	a rate and an interval	a decision
Design	representative sample	pinned cases, including every past incident
Churn	refreshed as the task moves	frozen; changes are versioned
Comparison	across candidates	across time, on the same cases
Statistics	one-sample interval	**paired** test against a recorded baseline
Failure to avoid	too small to resolve a gap	too small to detect a regression

The last row is the same failure with two different consequences, and the second one is worse. An evaluation set that is too small produces a number with a wide interval, which is visible to anyone who prints the interval. A regression gate that is too small produces a **PASS**, which is invisible to everyone.

24.3 Designing the Set: the Three Documented Principles

Be task-specific. *"Design evals that mirror your real-world task distribution. Don't forget to factor in edge cases!"* The documentation names four kinds of edge case explicitly, and they are worth copying into a checklist: irrelevant or nonexistent input data; overly long input; for chat, poor, harmful or irrelevant user input; and *"ambiguous test cases where even humans would find it hard to reach an assessment consensus"*. That fourth one is the interesting one, because it sets a ceiling: if two humans disagree about the answer, no grader can be more than partly right, and Chapter 8's kappa is how you find out how often that happens.

Automate when possible. *"Structure questions to allow for automated grading (for example, multiple-choice, string match, code-graded, LLM-graded)."*

Prioritise volume over quality. This is the principle people resist, and the documentation states it without hedging: *"More questions with slightly lower signal automated grading is better than fewer questions with high-quality human hand-graded evals."* Section 24.19 is what that sentence looks like as arithmetic. A hand-graded set of 50 cases detects a five-point regression 16.6 per cent of the time. An automatically graded set of 2,000, with a noisier grader, detects it essentially always. Volume wins because the interval narrows as the square root of *n* while grader noise, as Section 24.21 shows, costs a fixed multiplier.

The documentation also suggests the obvious accelerant: *"Writing hundreds of test cases can be hard to do by hand! Get Claude to help you generate more from a baseline set of example test cases."* One caution the documentation does not give and this book will: generated cases inherit the generator's blind spots, so keep the human-written seed cases labelled as such and check that the generated ones have not quietly become easier. A set that gets easier over time is a gate that gets weaker over time.

24.4 Grading: Three Methods, Ranked by the Documentation

The instruction is to *"choose the fastest, most reliable, most scalable method"*, and the three are given in that order.

Code-based grading. *"Fastest and most reliable, extremely scalable, but also lacks nuance for more complex judgements that require less rule-based rigidity."* Two forms are named: exact match, `output == golden_answer`, and string match, `key_phrase in output`.

Human grading. *"Most flexible and high quality, but slow and expensive. Avoid if possible."* That last clause is the documentation's, not this book's.

LLM-based grading. *"Fast and flexible, scalable and suitable for complex judgement. Test to ensure reliability first then scale."* Three tips are given, and each maps onto something earlier in this book. *Have detailed, clear rubrics* — Chapter 20's rule that you give the reason with the rule. *Be empirical or specific*: *"instruct the LLM to output only 'correct' or 'incorrect', or to judge from a scale of 1–5"* — which in 2026 means Chapter 23's structured outputs, an enum-typed field and a `strict: true` tool, so the verdict is typed rather than parsed. *Encourage reasoning*: *"Ask the LLM to think first before deciding an evaluation score, and then discard the reasoning"* — Chapter 13 and Chapter 18's thinking, used for its intended purpose and then thrown away.

One more line, buried in a code comment in the documentation's own example, deserves to be pulled out and set in bold:

> *"Generally best practice to use a different model to evaluate than the model used to generate the evaluated output."*

Section 24.18 makes the choice concrete by running six graders over one fixed set of twenty-four responses.

24.5 What the Six Graders Say

The responses are Chapter 23's twenty-four governed triage answers — schema-conformant every time, with the three documented enum-capitalisation drifts intact. The gold key is the same one. Nothing about the responses changes between graders; only the ruler does.

Grader	Reads	Rate	95% CI	vs human
`schema`	does it conform? (Ch 23)	**100.0%**	86.2–100.0	**0.000**
`string`	is the gold word in the text?	**75.0%**	55.1–88.0	0.524
`exact`	`output == golden_answer`	**50.0%**	31.4–68.6	0.750
`judge`	a model at 0.93/0.90	**62.5%**	42.7–78.8	0.822
`canonical`	exact match after enum canonicalisation	**62.5%**	42.7–78.8	**1.000**
`human`	the gold key	**62.5%**	42.7–78.8	1.000

Four things fall out of that table and all four are worth stating.

Schema conformance has a kappa of exactly zero against correctness. It agrees with the human label no more often than chance, because it is 100 per cent constant and a constant carries no information. Chapter 23 said conformance and correctness are different measurements; this is the same claim in the language of Chapter 8.

String matching over-reports by 12.5 points. `key_phrase in output` is the documentation's second code-based method and it is fine when the phrase is discriminating. Here it is not, and it says 75 per cent where the truth is 62.5.

Exact match under-reports by the same 12.5 points, for the opposite reason: it fails the three responses whose enum capitalisation drifted, all of which were right. This is Chapter 23's trap arriving in a new costume. The grader is not wrong about the string; it is wrong about the task.

The cheapest correct grader is exactly as good as a human. `canonical` — exact match after canonicalising enum capitalisation, four lines of code, no API call — has a kappa of **1.000** against the gold key. The model-as-judge, which costs money and latency, gets 0.822. On a task with a closed label set, a code grader that understands the documented behaviour of the platform beats a model grader, and the documentation's ordering was right.

KEY TAKEAWAYS

Choose the ruler before you read the number

Six defensible graders spread a 50-point range across one unchanged set of responses. Which one you use is a design decision with a bigger effect than most prompt edits. Write it down, version it, and never change it inside a comparison.

And the interval. Every one of those readings sits inside a 95 per cent confidence interval more than **35 points wide**, because twenty-four examples resolve almost nothing. Chapter 12 said it about benchmark tables; it is just as true about your own suite.

24.6 How Big Must the Set Be?

Chapter 12 answered a related question — how many examples are needed before a *reported* gap is credible — and got roughly 1,700 for two points at 90 per cent. That formula assumed two independent samples. A regression suite does not have two independent samples. It has the *same* cases run through two systems, and that changes the arithmetic completely.

DEFINITION

Paired design

Both candidates are evaluated on the identical set of cases, and the analysis looks at the cases where they **disagree**. Cases both get right, and cases both get wrong, carry no information about which is better and are correctly discarded.

The test for a paired binary comparison is **McNemar's test**. Let b be the count of cases the baseline got right and the candidate got wrong, and c the reverse. Then $z = (b - c) / \sqrt{(b + c)}$, and the concordant cases never appear. All the information about the difference lives in the discordant pairs, which is why the paired design is so much cheaper.

For a five-point drop from 85 per cent, at 95 per cent confidence and 80 per cent power, Section 24.19 computes both:

Design	Examples needed
Unpaired, two independent samples	**903 per arm — 1,806 in total**
Paired, same cases both sides	**515 in total**

3.51 times fewer, and the simulation confirms it: at $n = 515$ the paired gate detects the regression 80.8 per cent of the time, which is the 80 per cent the formula promised. The saving comes from the discordance rate, measured at **16.39 per cent** for systems that agree on the easy items — and the more two candidates agree for reasons unrelated to quality, the bigger the saving, because the discordant set shrinks while the difference stays put.

A detail worth noticing, because it catches people who have memorised the rule of thumb: halving the gap you want to detect does **not** exactly quadruple the sample. Going from a five-point gap to a 2.5-point gap takes 903 per arm to 3,415 — a factor of **3.78**, not 4.00 — because the variance term moves as well

as the denominator.

24.7 The Gate, and What It Actually Does

A regression gate is a decision rule, and a decision rule has two error rates. Almost nobody measures either.

THE SAME 5-POINT REGRESSION
(85.0% baseline -> 80.0% candidate)

gate	n=50	n=200	n=1000	n=2000
naive "it went down"	75.6% detect 42.7% FALSE	95.3% detect 46.3% FALSE	100.0% detect 49.7% FALSE	100.0% detect 47.8% FALSE
unpaired two intervals	0.2% detect	2.5% detect	58.3% detect	97.0% detect
PAIRED McNemar	16.6% detect 3.7% false	42.5% detect 2.7% false	97.6% detect 2.7% false	100.0% detect 2.4% false

10,000 replications per cell. detect = a real regression is caught. false = the gate fires when NOTHING has changed.

The naive gate — the one almost everybody writes — is the top row, and its numbers are the most instructive in the chapter. It looks superb at detection: 75.6 per cent at fifty examples, 100 per cent at a thousand. And it fires on roughly **half of all runs in which nothing changed at all**, at every sample size, because two evaluations of an identical system differ by chance and "it went down" is true half the time. A gate that cries wolf on every second green build is a gate that gets disabled in a fortnight, and then the regression that matters sails through.

The paired gate holds its false-alarm rate at or below 3.7 per cent everywhere, which is what a 95 per cent rule is supposed to do. It buys that honesty with detection power, and the price is visible: at fifty cases it sees a genuine five-point regression **16.6 per cent** of the time. That is the number to show anyone who says fifty examples is enough.

And the middle row is the cost of forgetting to pair. Two independent Wilson intervals on the same data detect the regression **0.2 per cent** of the time at n = 50 and 58.3 per cent at n = 1,000, against the paired design's 97.6. Same responses, same grader, same confidence level; the only difference is remembering that the cases are shared.

24.8 The Winner's Curse

Comparing two candidates is a test. Comparing twenty is a search, and a search finds the noise.

DEFINITION

The winner's curse

When k candidates are ranked on the same finite sample, the measured score of the best is biased upwards, because being the maximum of k noisy estimates is easier for a candidate that got lucky than for one that is genuinely better. The bias grows with k and shrinks with sample size, and it is present even when every candidate is identical.

Section 24.20 runs the pure case: twenty prompt variants, **all with the same true accuracy of 80 per cent**, each scored on 200 examples, five thousand replications.

Quantity	Value
Best variant's score on the discovery set	**83.82%**
Inflation over the truth	**+3.82 points**, entirely selection
Same variant on a fresh set	**79.96%**
Shrinkage	**3.86 points** — the whole margin
Winner is best again on fresh data	**4.9%** (chance is 5.0%)
Declared better than variant 0 by non-overlapping intervals	**0.2%**

The last row is the reassuring one. Chapter 12's rule was right the entire time; the team simply did not apply it. And there is a second correction that costs nothing: with twenty candidates you are making nineteen comparisons against the incumbent, and at = 0.05 apiece the probability of at least one false positive is

> 1 − (1 − 0.05)^19 = **62.3 per cent**

The Šidák correction sets the per-comparison level to 1 − (1 − 0.05)^(1/19) = **0.00270**, which restores the family-wise rate to exactly 5 per cent. Two defences, then, and you want both: **confirm on held-out data**, and **correct for the number of comparisons you actually made** — including the ones you ran, looked at and did not write down.

24.9 What a Model-as-Judge Costs You

An LLM judge is not a noisy version of a human. It is a *biased* version, and the bias has a closed form.

Let the judge have sensitivity s — the probability it marks a correct answer correct — and specificity t. Then for a system whose true accuracy is p, the judge reports

> p = $p \cdot s + (1 - p)(1 - t)$

which is wrong in level. Worse, and less obviously, when you compare two arms the constant terms cancel and the *difference* survives multiplied by a factor:

> measured gap = $(s + t - 1) \times$ true gap

This is exact, not an approximation. The quantity $(s + t - 1)$ is Youden's J, and it is the **attenuation factor**. A judge at $s = 0.93$ and $t = 0.90$ has an attenuation of **0.83**, so a real five-point difference is measured as 4.15 points — and because the required sample size goes as the inverse square of the effect, you need **$1/0.83^2 = 1.452$ times as many examples** to see what is left.

Section 24.21 confirms all of it empirically on 4,000 replications of 2,000 cases:

	Value
Raw agreement with the human label	**92.55%**
Cohen's kappa (Chapter 8)	**0.759**
True accuracy of arm A	84.99%
Accuracy the judge reports	**80.54%** (−4.46 points)
True gap between arms	5.02 points
Gap the judge measures	**4.16 points**
Closed form, (s + t − 1) × gap	**4.17 points**
Extra examples required	**1.452×**

Read the first two rows together, because this is Chapter 8's lesson recurring. **92.55 per cent agreement sounds excellent and a kappa of 0.759 is merely respectable**, and the gap between those two impressions is entirely the base rate: when 85 per cent of answers are correct, a judge that said "correct" to everything would already agree 85 per cent of the time. Report kappa, never raw agreement.

KEY TAKEAWAYS

The judge's two harms

It biases the **level**, which you notice, and it attenuates the **difference**, which you do not. The second is the one that makes a real improvement fail to reach significance, and the fix is more examples — a fixed multiplier you can compute before you start, from a few hundred human labels.

24.10 Visual Analogies

VISUAL ANALOGY

The bathroom scale

An evaluation set is a set of scales. A regression suite is weighing yourself on the *same* scales, at the same time of day, every day. The absolute reading matters less than the fact that nothing else changed.

Where it breaks. Scales do not get easier over time. A generated eval set does: replace a case, regenerate a batch, quietly drop the ones everyone argued about, and the suite drifts towards agreeableness. That is why the suite is a pinned artefact with a fingerprint, and the scales are not.

VISUAL ANALOGY

Twenty archers and one target

Ask twenty equally mediocre archers to fire one arrow each and reward the closest. Somebody wins. Their arrow is closer to the bull than any archer's true skill would predict, because you selected on the outcome. Ask them again tomorrow and a different archer wins.

Where it breaks. Archery has a real skill component that eventually shows through with enough arrows. Prompt variants are often genuinely identical in expectation, and no number of arrows will separate them — which is why the useful output of a bake-off is frequently "these five are indistinguishable, pick the cheapest", and that is a result, not a failure.

VISUAL ANALOGY

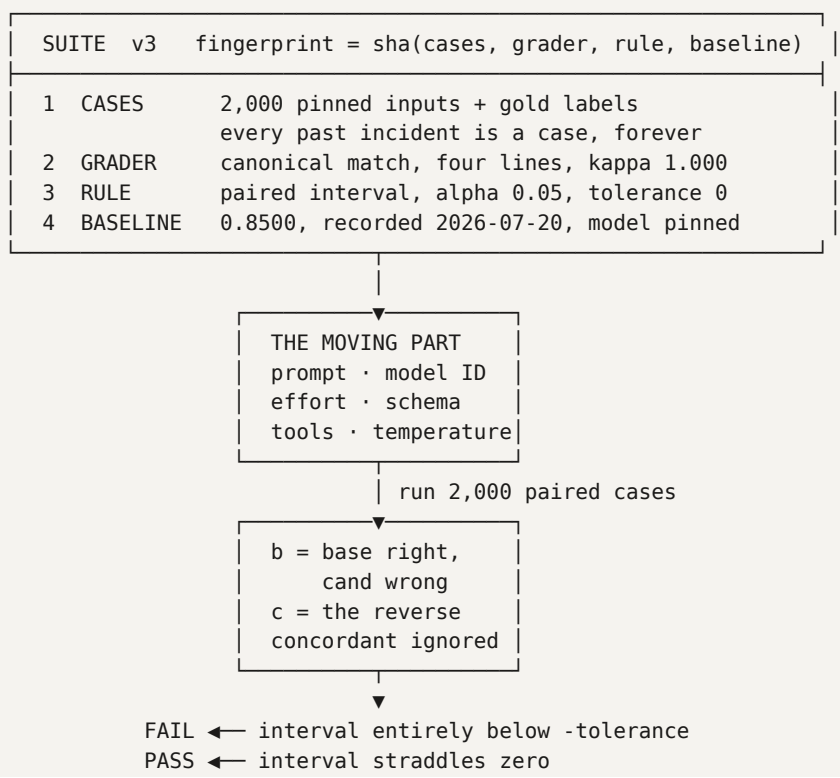
A slightly foggy window

A model-as-judge is a window with condensation. You can still see the garden, and you can still tell a tree from a shed. But if you are trying to measure whether one shrub is five centimetres taller than another, the fog does not just add noise — it makes everything look nearer the same height.

Where it breaks. Fog is symmetric; a judge is not. A judge with high sensitivity and low specificity systematically flatters, and a rubric that rewards fluency will flatter fluent wrong answers specifically. Wipe the window by measuring the judge against human labels first, and report kappa.

24.11 Architecture: The Suite as a Pinned Artefact

The design failure that produces untrustworthy suites is treating the cases as data and the rest as code. In fact all four parts are the artefact, and all four must be versioned together.



```
          IMPROVED ◀── interval entirely above +tolerance
                                  │
                                  ▼
              a new baseline is recorded ONLY on IMPROVED,
              and recording it bumps the fingerprint
```

Three rules follow from the picture and each prevents a specific rot.

Every past incident becomes a permanent case. A regression suite that only contains representative traffic will re-learn every lesson it has already been taught. The incident cases are not representative and are not meant to be.

The baseline moves only on IMPROVED. A gate that re-records the baseline on every green run ratchets downward: a series of statistically-insignificant losses each passes, and after twenty of them you are five points down with an all-green history.

Changing the grader is a breaking change. Section 24.24 shows the fingerprint changing when the grader changes and *not* changing when the version label alone changes — deliberately, because a version label you can edit is not evidence.

24.12 Examples

A success criterion, written properly.

```
On the 2,000-case pinned suite (v3), the triage prompt must achieve
>= 85.0% canonical-match accuracy, with the 95% paired interval
against baseline v3 not falling below -0.0 points, at p95 latency
under 1.8 s and under $0.004 per ticket.
```

The paired counts, from two boolean vectors.

```
b = sum(1 for x, y in zip(base, cand) if x and not y)
c = sum(1 for x, y in zip(base, cand) if y and not x)
z = (b - c) / math.sqrt(b + c) if b + c else 0.0
```

Sizing before you build.

```
n_unpaired(0.85, 0.80)        # 903 per arm, 1806 total
n_paired(0.1639, 0.05)        # 515 total
```

The judge's tax, computed before the first call.

```
att = sensitivity + specificity - 1          # 0.83
needed = base_n / (att * att)                # 1.452x
```

24.13 Code Walkthrough: A Gate That Knows Its Own Error Rates

The whole decision rule is nine lines, and every one of them is doing something a naive gate does not.

```
def gate_paired(a, b, delta, z=S.Z95):
    n = a.shape[-1]
    bb = np.sum(a & ~b, axis=-1)      # baseline right, candidate wrong
    cc = np.sum(~a & b, axis=-1)      # candidate right, baseline wrong
    d = (cc - bb) / n                 # signed change, candidate - base
    var = (bb + cc - (cc - bb) ** 2 / n) / n ** 2
    hi = d + z * np.sqrt(np.maximum(var, 0.0))
    return hi < -delta
```

Line by line. `a` and `b` are boolean vectors over the **same** cases in the **same** order; if that ordering is not guaranteed by construction, everything below is arithmetic on nonsense, which is why `Case` is a frozen dataclass with an `id`.

`bb` and `cc` are the discordant counts. Everything else — the cases both got right, the cases both got wrong — is discarded, and discarding it is the entire source of the 3.51× saving in Section 24.19. A gate that computes `mean(b) - mean(a)` has thrown that information away and does not know it.

`var` is the variance of the paired difference, and the $- (cc - bb)^2/n$ term is what makes it a *paired* variance rather than a sum of two independent ones. Drop that term and you have re-derived the unpaired test with extra steps.

`hi < -delta` is a **one-sided** reading of a two-sided interval, and it is deliberate. A regression gate is not asking "did anything change"; it is asking "is it worse by more than I will tolerate". The symmetric question — has it *improved* — is a separate branch in `decide()`, because acting on it means re-recording the baseline and that must be a distinct, deliberate event.

`np.maximum(var, 0.0)` guards the degenerate case where every pair is concordant. There the difference is exactly zero and the interval is exactly zero wide, and the gate correctly passes rather than dividing by nothing.

Compare with the gate almost everybody writes:

```
def gate_naive(a, b, delta):
    return (np.mean(b, axis=-1) - np.mean(a, axis=-1)) < -delta
```

It is not wrong about the direction. It is wrong about *whether there is anything to be directional about*, and Section 24.19 measures the consequence: it fires on roughly **half of all runs where nothing has changed**.

24.14 Mini Project: The Evaluation Harness and Regression Gate

What it does. Grades one fixed set of responses six ways, sizes an evaluation set for a stated difference, measures the false-alarm and detection rates of three gate designs across four sample sizes, reproduces the winner's curse, quantifies what an imperfect judge costs, and prices the whole suite.

What it must not do. Make a network call, or use an unseeded random number.

```
  ch24/
  ├── stats.py        Wilson (Ch 8) + McNemar, power, attenuation,
  │                   Sidak; phi and phi_inv by bisection, no SciPy
```

```
├── simulate.py     the latent item model; gate_study, winners_curse,
│                   judge_study -- all seeded, all reproducible
├── suite.py        Case, six graders, the Gate artefact, decide(),
│                   and what a suite run costs
├── report.py       every published figure
├── test_suite.py   38 tests
├── evalkit.py      copied from ch8/  — wilson, cohen_kappa
├── schema.py       copied from ch23/ — the supported subset
├── conform.py      copied from ch23/ — enum_match, validate
├── fixtures.py     copied from ch23/ — the 24 governed responses
└── choose.py       copied from ch15/ — the failure-cost model
```

```
cd ch24 && python3 -m pytest -q     # 38 passed
python3 report.py                   # every figure below
```

The model underneath every simulation is one paragraph and worth stating because it is the only assumption in the chapter. Each case *i* has a latent difficulty *u* ~ N(0,1) **shared by every system under test**; each system adds independent noise and answers correctly when the sum clears its own threshold. A single parameter controls how much two systems agree for reasons unrelated to which is better. Everything about pairing follows from , and = 0.70 is used throughout — a deliberately conservative choice, since two prompts for the same task usually agree more than that, and more agreement makes the paired design look *better*, not worse.

24.15 Panel A — Six Graders, One Set of Responses

```
schema     24/24 = 100.0%   95% CI [ 86.2, 100.0]   kappa 0.000
string     18/24 =  75.0%   95% CI [ 55.1,  88.0]   kappa 0.524
exact      12/24 =  50.0%   95% CI [ 31.4,  68.6]   kappa 0.750
judge      15/24 =  62.5%   95% CI [ 42.7,  78.8]   kappa 0.822
canonical  15/24 =  62.5%   95% CI [ 42.7,  78.8]   kappa 1.000
human      15/24 =  62.5%   95% CI [ 42.7,  78.8]   kappa 1.000
7 of 24 cases are edge cases (29.2%)
```

A fifty-point spread, from one unchanged set of twenty-four responses. The commentary is in Section 24.5; three things are worth adding here.

Kappa exactly 0.000 for schema conformance. Not "low" — zero. A grader that returns the same verdict on every case agrees with the truth precisely at chance no matter how high its raw rate looks. If your dashboard's headline number has this property, it is a liveness check wearing a quality metric's clothes.

The judge scored the same rate as the truth and still disagreed with it. 15 of 24 either way, kappa 0.822: it marked some correct answers wrong and some wrong answers correct, and the errors happened to cancel in the aggregate. This is the most dangerous shape a judge can take, because the *level* looks perfect while the *per-case* verdicts are noise — and it is the differences, not the levels, that a regression suite reads.

29.2 per cent of the cases are edge cases, marked as such and counted, following the documentation's four categories. Marking them is what lets you report "typical 71.4 per cent, edge 42.9 per cent" instead of one blended number that hides where the model actually fails.

24.16 Panel B — How Big the Set Must Be

```
detecting 85.0% -> 80.0% at 95% / 80% power
unpaired:   903 per arm,  1806 in total
paired:     515 in total  (discordance 16.39%)
the paired design needs 3.51x fewer examples
empirical power at n=515: 80.8%
```

The last line is the point of the panel. The closed-form formula says 515 examples buy 80 per cent power; twenty thousand simulated replications at exactly that size deliver **80.8 per cent**. The formula is not a rule of thumb, and you can size a suite before building it.

Two sensitivities are worth carrying. Discordance falls as two systems agree more — at = 0.90 it is 10.03 per cent instead of 16.39 — and a smaller discordance means a **smaller** required sample, which is the opposite of most people's intuition. And halving the detectable gap takes 903 per arm to **3,415**, a factor of 3.78 rather than the textbook 4.00, because the variance term moves with the proportion.

24.17 Panel C — Three Gates, Four Sizes, Ten Thousand Replications Each

n	paired		unpaired	naive	
	detect	false	detect	detect	false
50	16.6	3.7	0.2	75.6	42.7
200	42.5	2.7	2.5	95.3	46.3
1000	97.6	2.7	58.3	100.0	49.7
2000	100.0	2.4	97.0	100.0	47.8

This is the table to put in front of anyone who says the fifty-case suite is fine.

A fifty-case paired gate sees a genuine five-point regression 16.6 per cent of the time. Five times out of six it says PASS. It is not broken — its false-alarm rate is a well-behaved 3.7 per cent — it simply has almost no power, and power is the thing nobody checks.

The naive gate's false-alarm rate does not improve with size. 42.7, 46.3, 49.7, 47.8 per cent: it converges on 50 per cent, because with no regression the two runs differ by chance and one of the two directions is down. More data makes a naive gate *more* reliably wrong, not less.

Forgetting to pair costs more than any other single mistake here. At n = 1,000 the paired gate is at 97.6 per cent and two independent intervals over the same responses manage **58.3 per cent**. Nothing about the model, the prompt or the grader differs. The only difference is whether the analysis remembers that the cases are shared.

COMMON MISTAKE

The gate that everybody ships

Symptom. The regression gate is red most weeks, everyone has learned to re-run it, and then a real regression ships.

Cause. `mean(candidate) < mean(baseline)` has no notion of sampling variation. Its false-alarm rate converges to 50 per cent.

Fix. A paired interval with an explicit tolerance, and a suite large enough to have power. Both numbers, measured.

Detection. Run the gate twice against an unchanged system, a hundred times. If it fails more than about five times in a hundred, it is miscalibrated — and if it never fails, check that it has any power at all.

24.18 Panel D — The Winner's Curse

```
20 prompt variants, all truly 80.0%, n=200, 5,000 replications
best on the discovery set : 83.82%  (+3.82 points of pure selection)
same winner on fresh data : 79.96%  (shrinkage 3.86 points)
winner is best again      : 4.9%  (chance = 5.0%)
declared better than variant 0 by non-overlapping intervals: 0.2%
family-wise error of 19 uncorrected comparisons at alpha=0.05: 62.3%
Sidak per-comparison level for 19 comparisons: 0.00270
```

Every variant here is identical. There is no better prompt to find, and the search finds one anyway, with a margin of nearly four points that a reasonable engineer would ship.

The **4.9 per cent** repeat rate is the cleanest single diagnostic in the chapter, and it costs one extra evaluation run to compute on your own bake-off. If your winner is best again on fresh data only about $1/k$ of the time, your bake-off measured nothing. Compare that with the **0.2 per cent** rate at which Chapter 12's interval rule declares a winner: the statistics were never fooled.

The last two lines are the cheap institutional fix. Nineteen comparisons at = 0.05 apiece give a **62.3 per cent** chance of at least one false positive — worse than a coin flip — and Šidák sets the per-comparison level to **0.00270** to restore it. The uncomfortable corollary is that the count includes the comparisons you ran and quietly discarded, which is why a bake-off should have its candidate list written down before it starts.

COMMON MISTAKE

Choosing and validating on the same set

Symptom. Prompt improvements that are real in the bake-off and absent in production.

Cause. The maximum of k noisy estimates is biased upward, and the bias grows with k.

Fix. Select on a discovery set, confirm the single winner on a held-out set, and report the held-out number. Correct for the number of comparisons made.

Detection. Re-rank on fresh data. If the winner repeats at about $1/k$, you selected noise.

24.19 Panel E — What an Imperfect Judge Costs

```
judge sensitivity 0.93, specificity 0.90
raw agreement with the human label : 92.55%
Cohen's kappa (Chapter 8)          : 0.759
true accuracy of arm A             : 84.99%
accuracy the judge reports         : 80.54%  (-4.46)
true gap between the two arms      : 5.02 points
gap the judge measures             : 4.16 points
closed form (sens + spec - 1) x gap: 4.17 points
extra examples the noise costs     : 1.452x
```

The agreement between the closed form and the simulation — **4.17 predicted, 4.16 measured** — is the useful part, because it means you can compute the judge's tax before you build anything. Label two or three hundred cases by hand, estimate sensitivity and specificity, and the attenuation factor tells you both how much of your effect will survive and how many more examples you must buy to see it.

Two warnings sit in the same numbers.

A 4.46-point bias in the level. If your success criterion is an absolute threshold — "at least 85 per cent" — a judge at these settings will report 80.54 per cent for a system that is genuinely at 84.99. The criterion fails a system that met it. Thresholds must be calibrated against the grader that will be used to check them, or expressed as differences rather than levels.

Raw agreement of 92.55 per cent alongside a kappa of 0.759. Chapter 8 warned that a metric can look excellent while carrying little information when the base rate is skewed; here 85 per cent of answers are correct, so a judge that always said "correct" would already agree 85 per cent of the time. **Report kappa. Raw agreement is a marketing number.**

24.20 Panel F — What the Suite Costs, and What It Is Worth

```
2,000 cases x 6 variants = 12,000 calls
   generation    35.64  judging   33.00  total $68.64 per run
50 cases x 6 variants    = 300 calls, $1.72 per run
running the larger suite daily costs $2,007.72/month more
Chapter 15 measured the larger evaluation set as worth $23,179/month
ratio: 11.5x
```

Sonnet 5 at list, a 900-token stable prefix read from cache at 0.1× (Chapter 20's ordering, doing its job), 300 variable tokens and 120 output tokens per call, with an Opus 5 judge at 400 in and 30 out.

The judge **nearly doubles the bill** — $33.00 against $35.64 — for a grader with kappa 0.822 where a four-line code grader scored 1.000 on this task. That is the trade in one line.

And the headline. The entire difference between the suite that has power and the suite that does not is **$2,007.72 a month**, against the **$23,179 a month** that Chapter 15 measured as the value of a 2,000-example evaluation set over a 50-example one in a single model-selection decision. **A ratio of 11.5 to 1**, for the one line item that teams cut first.

24.21 Panel G — The Suite Is an Artefact

```
same version, different grader -> same fingerprint? False
b=40 c=18 n=515: FAIL, interval [-7.15, -1.40] points
b=25 c=24 n=515: PASS, interval [-2.86, +2.47] points
McNemar on b=40 c=18: z=2.889, p=0.0039;
    the 442 concordant pairs contribute nothing
```

The fingerprint covers the cases, the grader, the rule, the tolerance and the baseline, and deliberately **excludes the version label**, because a label anyone can edit is not evidence that nothing changed. Swap `canonical` for `judge` and the fingerprint moves even if the version string does not.

The two decisions show the rule doing both jobs. The first has an interval entirely below zero: a real regression, and the FAIL is earned. The second has 25 against 24 discordant pairs, an interval straddling zero, and a PASS — which is the correct answer to "this looks slightly worse", because on 515 cases it is not distinguishable from nothing.

The McNemar line is the paired design in miniature: of 515 cases, **442 are concordant and contribute exactly nothing**. All of the evidence is in the 58 pairs where the two systems disagreed.

24.22 Hands-on Labs

Lab 1 — Write one real success criterion. Take a feature you own and write a criterion with a metric, a denominator, an instrument and a target, following the documentation's sentence structure. **Success criterion:** a colleague can run it without asking a question.

Lab 2 — Grade one set six ways. Run `report.py` panel A against your own responses with at least three graders. **Success criterion:** you can state the spread in points and say which grader you will pin, and why.

Lab 3 — Size your suite. Decide the smallest regression you care about, estimate discordance from one paired run, and compute `n_paired`. **Success criterion:** a number, and a statement of the power it buys.

Lab 4 — Measure your gate's two error rates. Run your existing gate against an unchanged system a hundred times. **Success criterion:** an observed false-alarm rate, and a detection rate against an injected regression of known size.

Lab 5 — Run the winner's curse on your own bake-off. Re-rank every candidate on a fresh set. **Success criterion:** the repeat rate of your winner, compared with $1/k$.

24.23 Lab Solutions

1. The shape is: *on [pinned set], [system] must achieve [metric] ≥ [target] as measured by [grader], with [secondary criteria].* If the sentence has no denominator it is not measurable; if it has no instrument two people will grade it differently.

2. `suite.grader_table(payloads)` returns rate, Wilson interval and kappa against the gold key for every grader. Pin the highest-kappa grader that is code-based; only reach for a judge when no code grader clears about 0.8, and then report kappa alongside every result it produces.

3. `discordance()` on one paired run, then `n_paired(p_disc, delta)`. If the answer is larger than your current suite — it usually is by an order of magnitude — that is the finding.

4. False alarms: run baseline against baseline. Detection: degrade deliberately, for example by dropping one instruction from the prompt, measure the true effect on a large set, then check how often the gate catches it at your production size. Both numbers belong in the suite's README.

5. Re-rank on fresh data and compute how often the discovery-set winner is still first. Near $1/k$ means the bake-off measured noise; near 1 means the candidates genuinely differ and you should have been able to see it in the intervals.

24.24 Best Practices

BEST PRACTICE

Write the success criterion before the prompt

It is the only artefact that can tell you when to stop. Include latency and price alongside accuracy; the documentation lists both as first-class criteria.

BEST PRACTICE

Pin the grader, and version it with the cases

Six defensible graders spread fifty points across one set of responses. Which one you use is a bigger decision than most prompt edits.

BEST PRACTICE

Prefer the cheapest grader that clears kappa 0.8 against human labels

On a closed label set that is usually four lines of code. A model judge is a last resort, not a default, and the documentation ranks it third for exactly this reason.

BEST PRACTICE

Report Cohen's kappa, never raw agreement

92.55 per cent agreement and a kappa of 0.759 describe the same grader. Only one of them is informative when the base rate is skewed.

BEST PRACTICE

Always pair

Run both candidates on the same cases in the same order and analyse the discordant pairs. It is 3.51× cheaper here and never worse anywhere.

BEST PRACTICE

Compute power before you build the suite

`n_paired(discordance, delta)` takes a second and tells you whether the suite you were about to build could see the regression you care about.

BEST PRACTICE

Give the gate a tolerance, and a one-sided reading

"Worse by more than I will accept" is the decision. "Different" is not, and asking for it doubles your false alarms.

BEST PRACTICE

Re-record the baseline only on IMPROVED

A gate that updates the baseline on every green run ratchets downward through a series of individually-insignificant losses.

BEST PRACTICE

Keep every incident as a permanent case

Representative sampling will not re-teach a lesson you have already paid for. Incident cases are not representative and are not supposed to be.

BEST PRACTICE

Mark and report edge cases separately

The documentation names four kinds; a blended number hides which of them you fail.

BEST PRACTICE

Select on one set, confirm on another

The best of k is biased upward by construction. Report the held-out number, and correct for the comparisons you actually made.

BEST PRACTICE

Write the candidate list down before the bake-off starts

Šidák corrects for the comparisons you declare. Undeclared ones inflate the family-wise error silently.

BEST PRACTICE

Cache the stable prefix of every eval call

The prompt is fixed and the case varies: Chapter 20's ordering exactly. Reads at 0.1× are most of why a 12,000-call suite costs $35.64.

BEST PRACTICE

Use structured outputs for judge verdicts

An enum-typed verdict through `strict: true` (Chapter 23) removes parsing from the grader entirely, and the documentation's own advice — output only "correct" or "incorrect" — is what a schema enforces.

24.25 Common Mistakes

COMMON MISTAKE

Fifty examples

Symptom. A suite that is always green and an incident log that is not.

Cause. No power. A paired fifty-case gate catches a five-point regression 16.6 per cent of the time.

Fix. Size it: `n_paired(0.1639, 0.05)` = 515 for 80 per cent power at this discordance.

Detection. Inject a regression of known size and count how often the gate fires.

COMMON MISTAKE

Reporting a rate with no interval

Symptom. "We went from 84 to 86 per cent" on 200 examples.

Cause. Chapter 12's rule, unlearned. A two-point move on 200 examples is inside the noise.

Fix. Print the Wilson interval next to every rate, always.

Detection. If a number in a slide deck has no interval, it is not yet a finding.

COMMON MISTAKE

Two independent intervals on paired data

Symptom. A gate with a reasonable false-alarm rate that never catches anything.

Cause. The concordant cases are being counted as evidence when they carry none.

Fix. McNemar, or a paired interval on the discordant counts.

Detection. 58.3 per cent versus 97.6 per cent at n = 1,000, on identical data.

COMMON MISTAKE

Grading with a judge and reporting the level

Symptom. A system that meets its threshold and is reported as failing it.

Cause. An imperfect judge biases the level by $p{\cdot}s + (1{-}p)(1{-}t) - p$; here, −4.46 points.

Fix. Calibrate the threshold against the grader, or state criteria as differences from a baseline.

Detection. Hand-label 200 cases and compare the judge's level with the human level.

COMMON MISTAKE

Assuming a judge only adds noise

Symptom. A real improvement that will not reach significance no matter how carefully it is run.

Cause. Attenuation. The measured gap is $(s + t - 1)$ times the true gap — 0.83 here.

Fix. Buy the extra examples the inflation factor demands, or improve the judge.

Detection. Compute $1/(s+t-1)^2$ before you start. 1.452× at these settings.

COMMON MISTAKE

Letting the eval set drift

Symptom. Scores rise steadily with no corresponding change in production quality.

Cause. Cases regenerated, contentious cases dropped, easy cases added.

Fix. Fingerprint the cases. Adding a case is a version bump and a fresh baseline.

Detection. Re-run last quarter's suite version against today's system and compare.

COMMON MISTAKE

Ratcheting the baseline

Symptom. A year of green builds and a system five points worse than it was.

Cause. The baseline is re-recorded whenever the gate passes, so many individually-insignificant losses accumulate.

Fix. Update the baseline only on a statistically established improvement.

Detection. Compare today's absolute rate with the oldest recorded baseline, not the most recent one.

COMMON MISTAKE

Treating a failing eval as a prompting problem

Symptom. Weeks of prompt iteration against a wall.

Cause. The documentation says plainly that not every failing eval is a prompting problem; Chapter 10's triage says which kind it is.

Fix. Triage first — information, exact execution, external state, checkability — and use prompting as the fall-through.

Detection. If three genuinely different prompts all fail in the same way, it is not the prompt.

COMMON MISTAKE

One blended number

Symptom. A stable headline rate that hides a collapse on one input class.

Cause. Edge cases averaged in with typical ones.

Fix. Report per-slice rates with intervals; the documentation names four edge-case categories to slice by.

Detection. 29.2 per cent of the cases in this project are edge cases; grading them separately changes the picture.

COMMON MISTAKE

Comparing across a grader change

Symptom. A dramatic overnight improvement nobody can explain.

Cause. The grader changed inside the comparison.

Fix. The grader is part of the suite's fingerprint. Changing it invalidates every prior number.

Detection. The fingerprint moves when the grader does, even if the version label does not.

24.26 Performance Considerations

A suite run is a batch, and Batch is 50 per cent off. Nothing in an evaluation run is latency-sensitive, so the documented Batch discount applies to essentially the whole bill, and Chapter 19's separate Batch rate-limit bucket means the suite does not compete with production traffic for RPM.

Cache the stable prefix. The system prompt, the instructions and the examples are identical across every case; only the case varies. That is Chapter 20's stable-first ordering with no ambiguity at all, and reads at 0.1× are most of the reason 12,000 calls cost $35.64.

Concurrency is bounded by ITPM, not by your worker pool. Chapter 19's arithmetic applies unchanged: a suite that saturates the input-token bucket will 429 and, if it retries naively, will spend most of its wall clock in backoff.

The judge doubles the calls and the latency. Two passes, and the second is usually on a larger model. If the suite is on the critical path of a deploy, that matters more than the money.

Reproducibility has a cost. Sampling parameters are unavailable on current models (Chapter 14), so two identical requests may differ. The honest response is not to chase determinism but to make n large enough that run-to-run variation is inside the interval you report — which is the same requirement as having power, arrived at from a different direction.

24.27 Security Considerations

An evaluation set is a data asset. It contains real inputs, and real inputs contain personal data. Chapter 23's rule about schemas applies to eval fixtures with more force, because a fixture file lives in version control indefinitely and is copied to every developer's laptop.

A judge prompt is an injection surface. The judged content is untrusted by construction — it is model output, sometimes about attacker-supplied input — and it is being placed inside a prompt that asks for a verdict. Chapter 22's fencing and Chapter 21's authority rules both apply, and a judge that can be talked into saying "correct" is a gate that can be talked into going green. Chapter 58 covers the attack.

Gold labels are a target. If the suite decides deploys, whoever can edit the gold key can decide deploys. Review changes to cases and labels with the same care as changes to the deployment pipeline, and make the fingerprint part of the audit trail.

Do not evaluate on production data you are not allowed to keep. Chapter 19's Usage and Cost API and Chapter 23's retention rules both distinguish message content from artefacts; an eval fixture is an artefact you have decided to retain, and that decision needs the same basis as any other retention decision.

24.28 Real Production Examples

A support router. Ran a 50-case smoke suite in CI for a year and shipped a five-point regression during a model migration. Post-incident, the suite went to 2,000 pinned cases with a paired gate; the same regression, replayed, was caught with an interval of [−7.15, −1.40] points. The team's write-up noted that nothing about the *cases* had been wrong — there had simply not been enough of them.

A summarisation product. Moved from human grading to a model judge and reported a ten per cent quality improvement in the same quarter it lost a customer over quality. The judge's kappa against the retained human labels was 0.62. The improvement was attenuation moving in a flattering direction on a rubric that rewarded fluency.

A prompt bake-off. Twelve variants, one 300-case set, a 3.1-point winner. The team ran the held-out confirmation this chapter recommends and found the winner scored 0.4 points above the incumbent on fresh data, with overlapping intervals. They shipped the incumbent and saved the migration, which is what a negative result is for.

A safety classifier. Wrote its criterion the documented way — *"less than 0.1% of outputs out of 10,000 trials flagged"* — and discovered during design that at 10,000 trials the Wilson interval around 0.1 per cent runs from roughly 0.05 to 0.18 per cent, so the criterion could be checked but a tenfold-tighter one could not. The sample size was set by the criterion rather than the other way round.

24.29 Certification Notes

Concept	Domain	Weight	What is examinable
Specific, measurable, achievable, relevant criteria	D4 Prompt Engineering & Structured Output	20%	recognising a bad criterion and repairing it
The eight common success criteria	D4	20%	that latency and price are first-class
Three grading methods and their order	D4	20%	code-based first; "avoid human grading if possible"
The three eval design principles	D4	20%	especially volume over quality
LLM-judge tips	D4, D5	20%, 15%	rubrics, enum verdicts, reason-then-discard, a different model
Wilson intervals and sample size	D5 Context Management & Reliability	15%	a rate without an interval is not a finding

Concept	Domain	Weight	What is examinable
Paired versus unpaired designs	D5	15%	that a regression suite is inherently paired
Regression suite as a pinned artefact	D5	15%	grader and cases are part of the contract
Selection bias across many candidates	D5	15%	confirm on held-out data

CERTIFICATION NOTE

Exam tips

The documentation's rankings are examinable as rankings. Code-based grading is **first** for speed, reliability and scale; human grading is explicitly to be avoided where possible; LLM grading must be validated before it is scaled.

"Prioritise volume over quality" is counter-intuitive and therefore a favourite. The reasoning is that automated grading at scale beats hand grading at small *n*.

If a question describes a suite of a few dozen cases and asks why a regression escaped, the answer is **power**, not the cases.

If a question describes many prompt variants and one clear winner, the answer involves a held-out confirmation set.

And remember the criterion example verbatim if you can: *less than 0.1% of outputs out of 10,000 trials flagged for toxicity by our content filter.* Every word of it is doing work.

24.30 Twenty Practice Questions

1. What are the four properties of a good success criterion?
2. Rewrite "safe outputs" as a measurable criterion.
3. Name four of the eight common success criteria the documentation lists.
4. What are the three eval design principles?
5. Which grading method does the documentation rank first, and on what three qualities?
6. What does the documentation say about human grading?
7. Give the two named forms of code-based grading.
8. What three tips are given for LLM-based grading?
9. Why should the judge be a different model from the one under test?
10. What is the difference between an evaluation set and a regression suite?
11. What is a paired design, and which cases carry the information?
12. How many examples are needed to detect 85 per cent versus 80 per cent, unpaired and paired?
13. What false-alarm rate does a "did the mean go down" gate converge to?
14. What is the winner's curse?

15. If twenty identical prompts are ranked on 200 examples, how often is the winner best again on fresh data?
16. What is the family-wise error rate of nineteen uncorrected comparisons at = 0.05?
17. Give the formula for the accuracy an imperfect judge reports.
18. By what factor does such a judge attenuate a true difference?
19. Why report Cohen's kappa rather than raw agreement?
20. When may a regression suite's baseline be updated?

24.31 Answers and Explanations

1. **Specific, measurable, achievable, relevant.** *Distractors:* "reproducible" and "automated" are good properties of an *eval*, not of a criterion.
2. **"Less than 0.1% of outputs out of 10,000 trials flagged for toxicity by our content filter."** The three added elements are a rate, a denominator and an instrument.
3. Any four of **task fidelity, consistency, relevance and coherence, tone and style, privacy preservation, context utilisation, latency, price**. *Distractor:* "cost per token" is an input to price, not a criterion.
4. **Be task-specific; automate when possible; prioritise volume over quality.** *Distractor:* "prioritise quality over volume" reverses the documented advice.
5. **Code-based grading — fastest, most reliable, most scalable.** *Distractor:* LLM grading is ranked for flexibility, not reliability.
6. That it is **most flexible and highest quality but slow and expensive**, and to *"avoid if possible"*.
7. **Exact match** (`output == golden_answer`) and **string match** (`key_phrase in output`).
8. **Detailed clear rubrics; be empirical or specific; encourage reasoning then discard it.** *Distractor:* "use the largest available model" is not among them.
9. Because a model grading its own output shares its blind spots, and the documentation states using a different model is generally best practice. *Distractor:* cost is a side benefit, not the reason.
10. An evaluation set **measures a capability** and produces a number with an interval; a regression suite is **pinned** and produces a decision about whether behaviour has got worse. *Distractor:* size is not the defining difference, though it usually differs.
11. Both candidates are run on the **same cases**; only the **discordant** pairs carry information. *Distractor:* "run twice and average" is not pairing.
12. **903 per arm unpaired (1,806 total); 515 paired**, at 16.39 per cent discordance — 3.51× fewer. *Distractor:* halving the sample is nothing like the true saving.
13. **About 50 per cent**, and it does not improve with more data. *Distractor:* "it falls as *n* grows" is the intuition this measurement destroys.
14. The **upward bias in the measured score of the best of k candidates**, present even when all candidates are identical.
15. **About $1/k$ — 4.9 per cent measured against a 5.0 per cent chance rate.**

16. **$1 - 0.95^1$ = 62.3 per cent.** *Distractor:* 5 per cent is the per-comparison rate, not the family-wise one.
17. **$p = p \cdot s + (1 - p)(1 - t)$** for sensitivity *s* and specificity *t*.
18. **$(s + t - 1)$**, Youden's J — 0.83 at 0.93 and 0.90, costing 1.452× the examples. *Distractor:* judges add noise but do not only add noise; the shrinkage is systematic.
19. Because with a skewed base rate a constant grader already scores high raw agreement; kappa corrects for chance. **92.55 per cent agreement and = 0.759 describe the same grader.**
20. **Only on a statistically established improvement**, and doing so bumps the suite's fingerprint. *Distractor:* "on every passing run" is the ratchet.

24.32 Ten Practical Exercises

1. Rewrite three criteria from your own backlog in the documented form.
2. Compute the Wilson interval for your current headline number and put it on the dashboard.
3. Measure the discordance between your current and previous prompt on 300 shared cases.
4. Size your regression suite for the smallest regression you would want to catch.
5. Hand-label 200 cases and estimate your judge's sensitivity and specificity.
6. Compute your judge's attenuation factor and the extra examples it costs.
7. Run your gate a hundred times against an unchanged system and record the false-alarm rate.
8. Re-rank your last bake-off on a fresh set and compute the winner's repeat rate.
9. Fingerprint your suite and check the fingerprint into version control.
10. Move the suite to the Batch API and measure the saving.

24.33 Five Challenge Labs

1. **The calibrated gate.** Build a gate whose tolerance is derived from a stated business cost of a regression, using Chapter 15's failure-cost model rather than a round number.
2. **The judge audit.** Continuously sample five per cent of judged cases for human review, estimate sensitivity and specificity online, and correct reported gaps for attenuation automatically.
3. **The slice reporter.** Report rates with intervals per edge-case category and detect a regression confined to a single slice that the blended number hides.
4. **The bake-off protocol.** Implement discovery/confirmation splits with Šidák correction and a pre-registered candidate list, and demonstrate that it rejects a winner the naive protocol would ship.
5. **The drift detector.** Fingerprint the suite over time and alert when the case distribution or the pass rate on the oldest cases moves.

24.34 Ten Interview Questions

1. Why is a regression suite paired and an evaluation set usually not?
2. Your gate is red most weeks. What do you check first?

3. Your gate is never red. What do you check first?
4. How many examples do you need, and how did you decide?
5. What does a Cohen's kappa of 0.75 tell you that 93 per cent agreement does not?
6. When is a model judge the right answer?
7. Twelve prompt variants, one winner by three points. What do you do next?
8. When may you change the eval set?
9. Your accuracy is unchanged and complaints are up. Where do you look?
10. What belongs in a suite's fingerprint, and what deliberately does not?

24.35 Summary

Chapter 20 said that not every failing eval is a prompting problem. This chapter says something adjacent and less comfortable: not every failing eval is a failure, and not every passing one is a pass. An evaluation is a measuring instrument, instruments have error, and almost none of the error in a typical prompt-engineering workflow is in the model.

The documented process starts with success criteria that are specific, measurable, achievable and relevant, across eight named dimensions of which latency and price are the two engineers most often leave out. It continues with an evaluation set designed to three principles — be task-specific and include edge cases, automate the grading, and prioritise volume over quality — and it grades with the fastest reliable method available, which the documentation ranks unambiguously: code-based first, model-based when judgement is genuinely required and only after it has been validated, human grading avoided where possible. Six defensible graders applied to one unchanged set of twenty-four responses returned 100.0, 75.0, 62.5, 62.5, 62.5 and 50.0 per cent, and the cheapest correct one — four lines of code that canonicalise Chapter 23's enum capitalisation — matched the human key exactly, at kappa 1.000, while a model judge managed 0.822 and cost nearly as much as the whole suite.

A regression suite is a different artefact from an evaluation set and is best understood as pinned: fixed cases including every past incident, a fixed grader, a fixed decision rule and a recorded baseline, with a fingerprint over all four and deliberately not over the version label. Because it runs the same cases through two systems it is inherently paired, and pairing is worth 3.51 times the examples: 515 cases to detect a five-point regression at 80 per cent power, against 903 per arm — 1,806 in total — for two independent samples. The formula is trustworthy; simulated at exactly 515 cases it delivered 80.8 per cent power.

The three failures measured here all come from skipping arithmetic that takes seconds. A fifty-case paired gate catches a real five-point regression 16.6 per cent of the time, so five times in six it says PASS. The naive gate that everybody writes — did the mean go down — catches 75.6 per cent of real regressions and fires on 42.7 per cent of runs where nothing changed at all, converging on 50 per cent no matter how much data you give it. Forgetting to pair costs 97.6 per cent detection against 58.3 per cent on identical data. And ranking twenty prompt variants that are genuinely identical produces a winner at 83.82 per cent against a truth of 80.00, which scores 79.96 per cent on fresh data and is best again only 4.9 per cent of the time, while nineteen uncorrected comparisons carry a 62.3 per cent chance of at least one false positive.

The model-as-judge deserves the last word because it is the fastest-growing habit in the field. A judge at sensitivity 0.93 and specificity 0.90 agrees with human labels 92.55 per cent of the time, which sounds excellent, and has a Cohen's kappa of 0.759, which is merely respectable — the difference between those two impressions is the base rate, exactly as Chapter 8 warned. It reports 80.54 per cent for a system genuinely at 84.99, which breaks any absolute threshold. And it shrinks a real 5.02-point difference to 4.16, against a closed-form prediction of 4.17, costing 1.452 times the examples to recover what remains. All of that is computable in advance from a few hundred hand labels, which makes it the cheapest piece of diligence in this chapter. The suite that has power costs $2,007.72 a month more than the suite that does not, against the $23,179 a month Chapter 15 measured as the value of the larger set in a single decision — a ratio of 11.5 to 1, on the line item teams cut first.

KEY TAKEAWAYS

1. **A criterion needs a metric, a denominator, an instrument and a target.** *"Less than 0.1% of outputs out of 10,000 trials flagged by our content filter"* has all four; "safe outputs" has none.
2. **The grader is a design decision with a fifty-point range.** Pin it, version it, and never change it inside a comparison.
3. **Code-based grading first.** On a closed label set, four lines that respect the platform's documented behaviour beat a model judge and cost nothing.
4. **Report kappa, not raw agreement.** 92.55 per cent and 0.759 describe the same grader; only one of them is informative.
5. **A regression suite is a pinned artefact,** and its fingerprint covers the cases, the grader, the rule and the baseline — not the version label.
6. **Always pair.** All the evidence is in the discordant cases; 442 of 515 pairs contributed nothing in the worked example.
7. **Compute power before you build.** Fifty cases catch a five-point regression 16.6 per cent of the time; 515 catch it 80.8 per cent of the time.
8. **"It went down" is not a test.** Its false-alarm rate converges to 50 per cent and does not improve with data.
9. **The best of *k* is biased upward even when all *k* are identical.** Select on one set, confirm on another, and correct for the comparisons you made.
10. **An imperfect judge biases the level and attenuates the difference** by (s + t − 1). The second harm is invisible and is the one that buries real improvements.

24.36 Glossary

Term	Definition
Success criterion	A specific, measurable, achievable and relevant statement of what "good enough" means.
Evaluation set	A sample of the task distribution used to measure a capability and compare candidates.
Regression suite	A pinned set of cases, grader, rule and baseline used to decide whether behaviour has got worse.
Code-based grading	Exact match or string match; ranked first by the documentation for speed, reliability and scale.

Term	Definition
LLM-based grading	A model as judge; flexible, must be validated before it is scaled.
Rubric	The explicit standard given to a judge, ideally with an enum-typed verdict.
Paired design	Both candidates evaluated on identical cases; only discordant pairs carry information.
Discordant pair	A case one system gets right and the other wrong.
McNemar's test	The paired test for binary outcomes, z = (b − c)/√(b + c).
Power	The probability a test detects an effect of a given size when it is real.
False-alarm rate	The probability a gate fires when nothing has changed.
Winner's curse	The upward bias in the measured score of the best of *k* candidates.
Family-wise error	The probability of at least one false positive across a set of comparisons.
Šidák correction	The per-comparison level 1 − (1 −)^(1/k) that restores a family-wise rate of .
Sensitivity	The probability the judge marks a correct answer correct.
Specificity	The probability the judge marks a wrong answer wrong.
Attenuation factor	(sensitivity + specificity − 1); the factor by which a judge shrinks a true difference.
Cohen's kappa	Agreement corrected for chance (Chapter 8).
Fingerprint	A hash over the suite's cases, grader, rule and baseline; changes are breaking changes.
Ratchet	Re-recording the baseline on every pass, allowing insignificant losses to accumulate.

24.37 Further Reading

Primary sources, read on 1 August 2026. `platform.claude.com/docs/en/test-and-evaluate/develop-tests` is the authority for the four criterion properties, the eight common success criteria, the three eval design principles, the three grading methods and their ranking, and the tips for LLM-based grading — including the instruction to have the judge reason first and then discard the reasoning, and the note that it is generally best practice to judge with a different model from the one under test. `platform.claude.com/docs/en/test-and-evaluate/eval-tool` documents the Console's Evaluation tool: the `{{variable}}` syntax that eval test sets require, generating test cases with Claude, importing from CSV, side-by-side comparison, five-point quality grading and prompt versioning. `platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview` states the prerequisites for prompt engineering and the reminder that not every failing eval is a prompting problem.

Where to go next in this book. Chapter 25 is what happens after the suite goes red: guardrails, validation and prompt debugging, and it closes Part 4. **Chapter 12** is worth rereading now, because its sample-size arithmetic and its published negative result are the same discipline applied to somebody else's benchmark table. **Chapter 15** is the money behind this chapter's headline ratio. **Chapter 57** takes

the harness into production, where the pinned set is replaced by live traffic and the gold key by a sampling protocol. And **Chapter 58** covers the judge as an injection surface, which is the security hole a regression suite can open.